SOURCE-LEVEL GUIDE · FLASHINFER × CAKE COMPILER

FlashInfer 与 CAKE：从原理到源码

一份为“想读懂、想贡献、想定制 FlashInfer”准备的导读：它解决什么问题，JIT 和 kernel 体系怎么搭起来，以及占了仓库绝大部分代码量的 CAKE：编译器在哪、Cake IR 长什么样、产物怎么进仓库、一次调用怎么跑到 GPU 上。

flashinfer-ai/flashinfer @ 555dd84

怎么读这份导读

这份导读写给有系统经验、第一次接触 FlashInfer 的工程师。读完你应该能做到：说清楚 FlashInfer 的一次调用怎么变成 GPU 上的 kernel；知道一个 `backend="cake"` 的 kernel 从哪来、怎么被选中、能不能改；判断自己想做的事该走哪条路。

> **注意 · 先说最重要的结论：CAKE 编译器不在这个仓库里**
>
> 你要找的 CAKE 编译器（Cake IR、verifier、cost model、codegen、agent harness）**不在 FlashInfer 仓库里，也不在 `flashinfer-ai` 组织的公开仓库列表里**。我在 GitHub 上也没有搜到官方实现，第三方项目 cake-ir 的 README 同样把官方实现称为未发布。仓库里有的是它的**导出产物**：生成的 CUDA、host binding、构建清单、路由表。代码里没有 `import cake`，`requirements.txt` 也不依赖它。
>
> 所以这份导读把 CAKE 拆成两半讲：编译器内部靠 CAKE 论文，产物和运行时靠固定版本的源码。论文的说法和源码的说法不会混在一起。

CAKE 相关章节里，每个说法都标了证据来源：

标记	含义	可信度
【论文】	来自 arXiv 2608.12629v1（2026-08-12），作者来自 NVIDIA 和 CMU	作者自述，部分内部设计论文明确不公开
【仓库】	在固定版本的源码里读到，旁边有链接	已逐条校验
【推断】	根据产物特征反推，没有直接证据	可能有误，用时请自行核实

全文顺序：

1. **全景**：问题、目标、限制，以及 FlashInfer 的整体结构。
2. **调用主干**：一次 `single_decode_with_kv_cache` 穿过 API、JIT、kernel 的完整路径。
3. **JIT 与分发**：JitSpec、架构目标、缓存、AOT、wheel。
4. **kernel 版图**：attention、GEMM、MoE、通信，以及多后端怎么选。
5. **CAKE 是什么**：它要解决的问题，以及编译器和仓库的边界。
6. **Cake IR**：语言设计，并用真实生成代码对照讲解 IR 概念。
7. **Harness 与进化循环**：verifier、cost model、agent 工作流、从单个 shape 到整个库。
8. **产物怎么进仓库**：五种交付形态、`build.json`、加载器、deslop。
9. **运行时**：路由、窗口、grid 规则、运行期 cost model。
10. **验证与可信度**：测试分层、确定性、已知限制。
11. **端到端生命周期**：从 Cake IR 到 GPU 上的一次反向传播。
12. **开源许可证、贡献与定制、源码索引**。

只关心 CAKE 的读者可以读完第 1 章后直接跳到第 5 章。

要点 · 固定版本

- FlashInfer：`flashinfer-ai/flashinfer` @ `555dd84`（2026-11-06，版本 `0.7.3`）
- CAKE 论文：arXiv 2608.12629v1
- 这个仓库变化极快：当天的 commit 还在给 CAKE 训练包改名。所有链接都固定在上面的 SHA，读最新代码时行号和名字可能已经变了。

全景：问题、目标、限制

没有 FlashInfer 时有什么问题

LLM 推理的大部分 GPU 时间花在几类 kernel 上：attention、线性层（GEMM）、MoE、通信。框架作者面对的问题不是“写不出来”，而是下面这些：

环节	问题	FlashInfer 的回答
组合爆炸	dtype × head_dim × 架构 × KV 布局 × 位置编码，预编译全部组合装不下	默认 JIT：用到哪个组合才编译哪个，缓存 `.so`
硬件代际	Hopper 的 WGMMA、Blackwell 的 `tcgen05` + TMEM、SM120 没有 TMEM，同一算子要重写多遍	按架构分别实现，同一个 Python API 背后多个 backend
专家 kernel 难复现	warp 分工、异步 barrier、内存层级的选择决定了最后 20% 性能，新模型一出来就要重做	CAKE：让 agent 在编译器辅助下生成 kernel
新模型节奏	DeepSeek、Kimi、Qwen、MiniMax 的新算子几周就要支持	实验轨道 `flashinfer/experimental/`，加上 CAKE 的批量产出

README 的定位是“a library and kernel generator for inference”（`README`）。这个定位在 2026 年有了新的含义：看代码量，FlashInfer 现在主要是一个**生成 kernel 的集散地**。

现在的仓库有多大

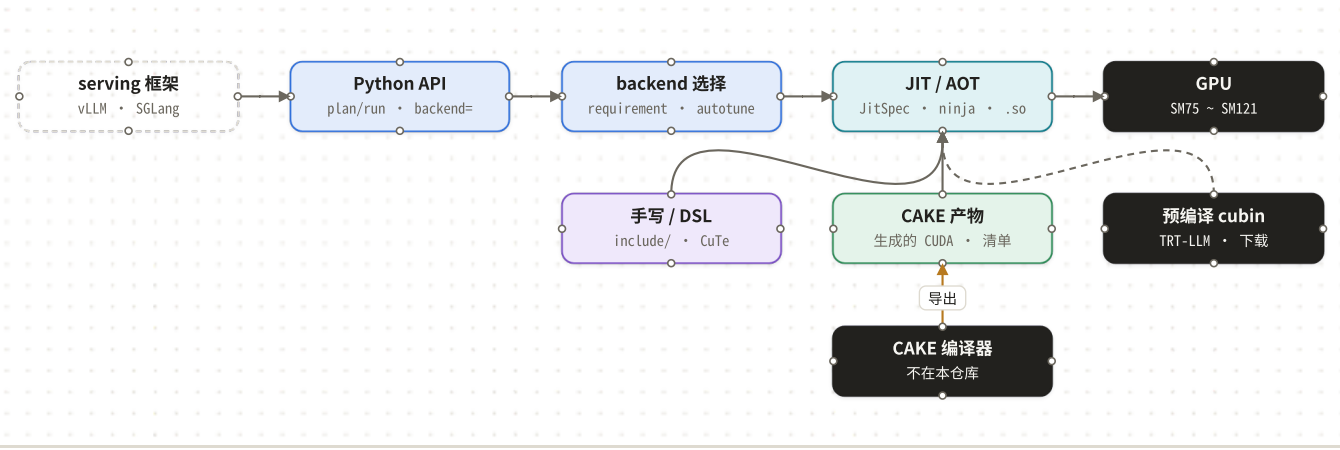
下面的数字是我在固定 SHA 上统计的。统计口径是路径名是否含 `cake`，所以 `deepgemm_*` 这类同样由 CAKE 生成、但路径不含 `cake` 的目录算在“其他”里，实际比例更高。

指标	数值
被跟踪的文件	18,309
路径含 `cake` 的文件	11,854（65%）
含 “Generated by Cake” 标记的文件	4,610
C/CUDA 代码行：路径含 `cake` / 其他	约 1158 万 / 约 72 万
Python 代码行：路径含 `cake` / 其他	约 131 万 / 约 312 万
`cake-build/1` 清单 `build.json`	54 个，登记 5,349 个库
`CakeProgram` 常量	449 个，分布在 32 个加载器

这张表决定了读这个仓库的方式：**不要试图读生成代码**。手写的核心不大，读懂主干和几个模式之后，生成代码按需查就够了。

一张总图

全景：FlashInfer 在 serving 栈里的位置



深色方框是仓库外的东西。CAKE 编译器和仓库之间只有一条单向的“导出”边：FlashInfer 运行时完全不依赖它。支持的架构见 `README 的 GPU 表`，版本打包见 `pyproject.toml`。

要求 / 目标 / 限制

项目	FlashInfer 核心	CAKE 产物层
核心目标	给 serving 框架提供统一 API、最快的 kernel	用 agent 批量产出超过专家水平的 kernel
功能要求	attention、GEMM、MoE、采样、通信；prefill/decode；FP8/FP4 量化	同样的算子，再加上反向传播（训练）和融合 kernel
规模压力	参数组合爆炸；每代硬件一套 ISA	单个算子数百个程序，每架构一份
外部限制	CUDA Graph、`torch.compile` 兼容；安装包不能太大；包目录可能只读	下游要能脱离 FlashInfer 构建；浮点语义不能被 fast-math 改掉
关键手段	JIT + 缓存；`@backend_requirement`；plan/run	内容寻址的文件名、清单校验、路由表、运行期 cost model
许可证	Apache-2.0，含第三方例外	生成代码带 Apache-2.0 头；编译器未公开

后面每一章都在解释这张表的某一格是怎么落地的。

调用主干：一次 decode 怎么跑到 GPU 上

职责划分

项目用三层把“框架”和“kernel”隔开：

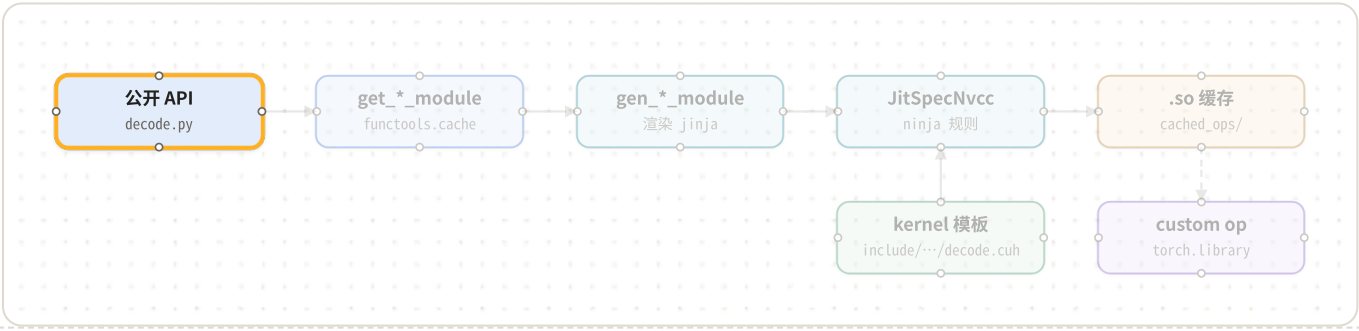
层	目录	规则
kernel 模板	`include/flashinfer/`	只收裸指针，**不准 include Torch 头文件**（`Framework Separation`）
绑定	`csrc/`	通过 TVM-FFI 接收张量、做校验、发起 launch
Python	`flashinfer/`	公开 API、参数校验、backend 选择、JIT 模块生成

这里的绑定层用 TVM-FFI 而不是 pybind。好处是 ABI 跨语言，kernel 不绑死 PyTorch（`TVM-FFI 说明`）。CAKE 生成的 binding 走的也是同一套接口。

一条真实路径

README 里最短的例子是 `flashinfer.single_decode_with_kv_cache(q, k, v)`。下面按步骤走一遍。

▶ single_decode_with_kv_cache 的第一次调用 1 / 6



1. 进入公开 API

`single_decode_with_kv_cache` 校验形状和 dtype，选后端。走默认路径时，它用参数元组去要模块：`get_single_decode_module(q.dtype, k.dtype, q.dtype, head_dim, head_dim, 位置编码, 滑窗, soft_cap)`。

2. 内存缓存

`get_single_decode_module` 有 `@functools.cache`：同一组参数在进程内只构建一次。这是两级缓存的第一级。

3. 生成规格

`gen_single_decode_module` 计算唯一 URI，把 jinja 模板渲染成类型特化的配置（`配置模板`），源文件写进可写的生成目录，不碰包目录。

4. 构建

`gen_jit_spec` 拼好 nvcc 参数（默认带 `-use_fast_math`，`core.py`）。`build_and_load` 在文件锁里写 `build.ninja` 并跑 ninja。头文件依赖由 ninja 跟踪，所以改了 `.cuh`，下次运行自动重编。

5. 加载并注册

`.so` 通过 `tvm_ffi.load_module` 加载，`module.run` 再包成 `torch.library` custom op（`register_custom_op`），这样 `torch.compile` 能看见它的副作用。

6. 进入 kernel

launch 到 GPU 的是 `SingleDecodeWithKVCacheKernel`。多数 serving 路径用的是批量版本 `BatchDecodeWithPagedKVCacheWrapper`，结构相同，只是多了 plan 这一步。

核心抽象

抽象	作用	位置
`JitSpec`	kernel 模块的生命周期：`try_load` / `build` / `load`	`core.py`
`CompilationContext`	决定编译哪些 CUDA 架构	`compilation_context.py`
plan/run 包装类	`plan()` 做一次调度，`run()` 只 launch，适合 CUDA Graph	`decode.py`
`paged_kv_t`	分页 KV cache 的设备侧视图	`page.cuh`
`@backend_requirement`	声明每个后端的能力，并做 `auto` 选择	`utils.py`
`@flashinfer_api`	日志、trace、公开 API 登记	`api_logging.py`

注意 · 容易误解的地方

- **两级缓存不是同一件事。** `functools.cache` 只在进程内；磁盘上的 `.so` 由 ninja 判断是否过期。`try_load` 只把 AOT 产物当作可信，JIT 路径的新鲜度交给 ninja。
- **CAKE 走的不是这条 `gen_*_module` 流程。** 它不渲染模板，直接编译已经生成好的单一 `.cu`，但最后同样落到 `gen_jit_spec`。

JIT 与分发

难点

JIT 要同时满足几个互相牵制的条件：多进程并发时不能重复编译；包目录可能只读；同一台机器可能插着不同架构的 GPU；离线环境不能现场编译。

生命周期

`build_and_load` 是所有工具链共用的模板方法：先 `try_load`；拿不到就进文件锁，再查一次（别的进程可能刚编完）；设置了 `FLASHINFER_DISABLE_JIT` 就抛 `MissingJITCacheError`；否则 build 再 load。

子类各管一种工具链：`JitSpecNvcc` 管 nvcc/ninja，`JitSpecCuteDsl` 管 CuTe DSL。可选的 TIRx 后端有自己的私有 spec（`三层结构说明`）。

目录规则很严格：

目录	可写	用途
`FLASHINFER_GEN_SRC_DIR`	是	渲染出的源文件
`FLASHINFER_JIT_DIR`	是	编译出的 `.so`
`FLASHINFER_CSRC_DIR`	否	只读的模板和源文件
`FLASHINFER_AOT_DIR`	否	预编译产物

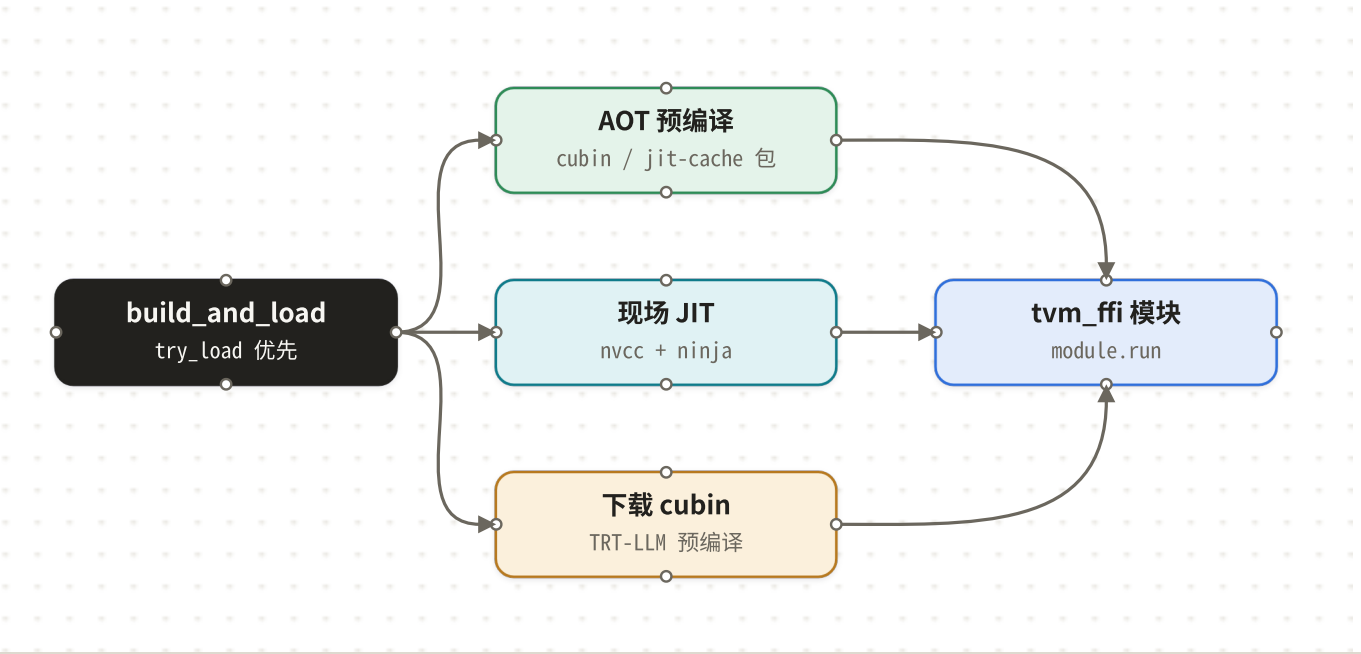
这些路径定义在 `env.py`。

架构目标

`CompilationContext` 优先读 `FLASHINFER_CUDA_ARCH_LIST`，否则探测本机所有 GPU。JIT 模块用 `supported_major_versions` 声明自己能编译的 SM 版本。SM107 有个细节：CUTLASS 不一定认识它，`cutlass_supports_sm107` 要同时确认 CUTLASS 头文件和 nvcc 都支持。

三条分发路径

同一个 kernel 的三种到达方式



`flashinfer-python` 只含源码；`flashinfer-cubin` 带预编译二进制；`flashinfer-jit-cache` 是 shim，依赖按架构构建的 `provider` 包。多架构场景由 `_HeterogeneousAOTModule` 按当前设备选模块。下载地址由 `FLASHINFER_CUBINS_REPOSITORY` 决定。

AOT 的注册集中在 `aot.py`。实验性代码不准注册到这里（`实验策略`）。但稳定区的 CAKE 模块可以注册：`aot.py` 里能看到 `gen_cake_fmha_compat_module`、`gen_cake_kda_decode_module` 这样的条目。

开发时的好处

改 `.cuh` 不用重装，下次调用自动重编。清缓存用 `rm -rf ~/.cache/flashinfer/`，看编译过程用 `FLASHINFER_JIT_VERBOSE=1`。完整列表在 `Quick Reference`。

kernel 版图与后端选择

主要家族

家族	Python 入口	设备代码	说明
Attention	`decode.py`、`prefill.py`	`attention/`	分页 KV cache、MLA、cascade、POD
GEMM	`gemm_base.py`	`gemm/`	BF16、FP8、NVFP4/MXFP4、分组
融合 MoE	`fused_moe/`	`csrc/fused_moe/`	路由 + 专家 + 累加
专家并行 MoE	`moe_ep/`	`kernel_src/`	Split 和 Mega 两种模式
通信	`comm/`	`comm/`	AllReduce、MNNVL、DCP all-to-all
线性注意力	`kda.py`、`gdn_prefill.py`	`csrc/kda/`、`csrc/gdn/`	KDA、Gated DeltaNet，有递归状态
采样	`sampling.py`、`cake_sampling.py`	`sampling.cuh`	不排序的 top-k/top-p

同一个算子，多个 kernel 来源

这是理解仓库的关键。下面这些实现可以同时存在，通过 `backend=` 参数区分：

来源	写法	例子
手写 CUDA 模板	C++ 模板，jinja 特化	`fa2`、`Hopper FA3`
CUTLASS	CUTLASS 的 C++ 模板	`fp8_gemm_cutlass`
CuTe DSL	Python DSL，运行时编译	`cute_dsl/`
Prims-TS	任务调度式 DSL，已 vendor	`prims_ts/`
TRT-LLM cubin	下载预编译二进制	`trtllm-gen`
cuDNN	NVIDIA 库	`cudnn/`
CAKE	生成的 CUDA	`backend="cake"`

选择逻辑

`@backend_requirement` 给一个函数登记每个后端的检查函数和 `heuristic_func`。调用时校验后端是否支持这个形状和架构；`backend="auto"` 时跑启发式；开了 autotune 就在候选里实测。以 `bmm_bf16` 为例，`cake` 是它的一个后端，登记的检查函数是 `_cake_bmm_bf16_requirement`，而 docstring 明确写着 `"cake"` 是显式才能选的（`gemm_base.py`）。

Attention 老路径的选择在 `determine_attention_backend`。autotuner 在 `autotuner.py`，已知不合法的组合由 `TacticsBlocklist` 提前过滤。

实验轨道

`实验策略` 区分两件事：实验 **API** 放在核心里，用 `@flashinfer_experimental_api` 标记；实验 **backend** 放在 `flashinfer/experimental/`。规则是：

- 显式调用不需要环境变量，只警告一次。
- `backend="auto"` 不会选实验后端，除非设置 `FLASHINFER_ALLOW_EXPERIMENTAL_AUTO_BACKENDS=1`。
- 实验代码只走 JIT，不进 AOT。
- 核心模块不准在顶层 import `flashinfer.experimental`（`实验目录规则`）。

CAKE 的很多包都在这条轨道上，这也解释了为什么 `experimental/` 下有 28 个 `cake_*` 目录，外加一个 `cake/` 命名空间。

CAKE 是什么：问题与边界

它要解决的问题

【论文】 CAKE 的全称是 *Compiler–Agent Co-Design for Frontier Kernel Evolution*，第一作者也是 FlashInfer 的作者之一。论文认为有两条路一直各走各的：

- **kernel agent** 把编译器当黑盒。它们改进提案和搜索，但环境只回报编译错误、对错、总耗时。这些信号说不清“是哪个程序决策导致了同步失败或流水线停顿”。
- **编程语言** 不是为 agent 设计的。tile 级 DSL 把 warp 分工、barrier 编排、内存层级藏起来，而这些正是专家 kernel 和普通 kernel 的差别；底层 DSL 暴露了这些控制，却要求掌握一套 layout 演算，agent 容易出错，出错了也难定位。

CAKE 的回答是三个承诺：

1. **agent 改的是有类型的 IR，不是裸 CUDA。** 硬件决策在生成代码之前就能被检查。
2. **编译器返回定位到具体程序位置的诊断，不是一个通过/失败的比特。** 便宜的分析先筛掉候选，不浪费 GPU 时间。
3. **harness 本身也在进化。** 反复出现的失败变成新的 verifier 规则、校准任务或 IR 原语，而且要过语料回归测试。

名字的由来和时间线

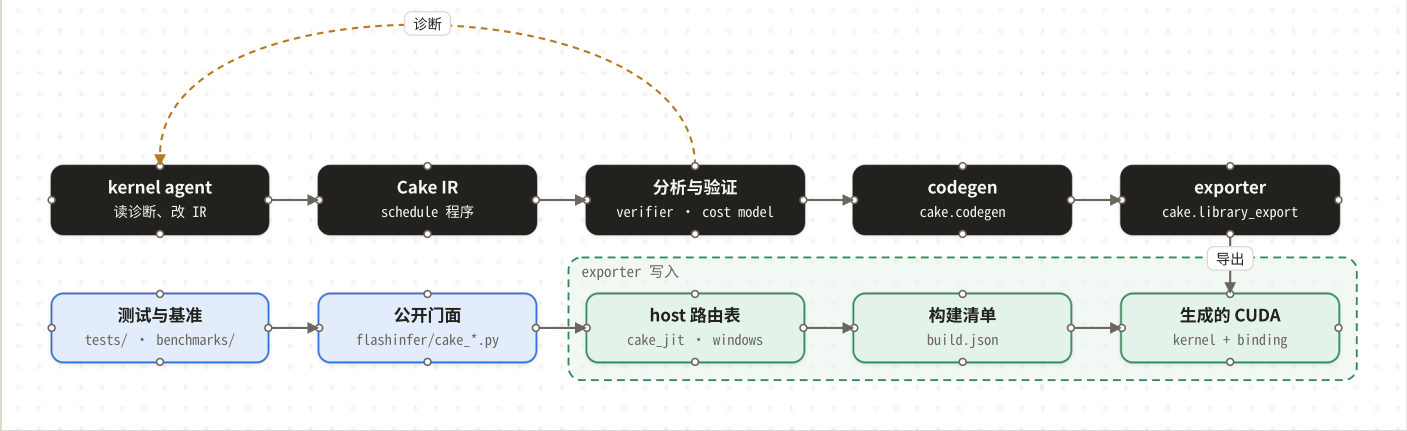
时间	事件	来源
2026-07-29	进度追踪 issue 开张，自述“CAKE harness + claude/codex agent”	#4254
2026-08-03	首批 CAKE kernel 之一合并：B200 上的 KDA prefill	#4262
2026-08-12	论文上线，列出 4 个 upstream PR	arXiv
2026-10-15	下游用户要求 CAKE 包能脱离 FlashInfer 构建	#6553
2026-10 起	“deslop”系列：清理生成代码的重复和死代码	示例 #6635
2026-11-06	标题含 cake 的 PR 累计 2,191 个（含未合并，GitHub 搜索结果）	GitHub 搜索

【推断】 编译器早期的内部名字很可能叫 Loom。证据有三处：#4274 的描述写着 “CAKE-generated Loom schedules”；`cake_ssd_combined.py` 的注释引用了 `loom/examples/weave/...`；GDN 的 README 提到生成器里的 `loom_` 前缀在导出时被映射成 `cake_`（`README`）。

编译器和仓库的边界

这张图是全文最重要的一张。深色部分你在 FlashInfer 里看不到。

CAKE 的边界：哪些在仓库里，哪些不在



下排箭头表示“谁引用谁”。绿框是 exporter 写入的导出物，**不要手改**，不少文件明确写着不要编辑（如 `cake_build.py`）。蓝色是人工维护的部分。深色节点的名字部分来自产物：`cake.codegen` 见 `device helper`，`cake.library_export.v5` 见 `清单 schema`。

能从产物里确认的编译器痕迹：

痕迹	说明	证据
Python 包名 `cake.codegen`	生成 device helper 的模块	`helper 文件头`
`cake.examples.cake.*`	kernel 的 IR 源文件放在 examples 下	`plan 文件`
`cake.library_export.v5`	导出格式，已迭代到第 5 版	`schema 常量`
`generator.cake_revision`	每个包记录生成它的编译器 commit	`build.json`（如 `rmsnorm`）
`CAKE-956` 之类的编号	内部工单	`cost model 注释`

54 个清单记录了 40 多个不同的 `cake_revision`，渲染时间从 2026-10-20 到 11-06。也就是说，各个包是独立、频繁地从编译器重新导出的。

类比 · 仓库收到的是蛋糕，不是烤箱

- **烤箱** = CAKE 编译器和 harness，在 NVIDIA 内部。
- **配方** = Cake IR schedule，同样不公开。
- **蛋糕** = 生成的 `_kernel.cu` 和 `_binding.cu`，就是你在仓库里看到的。
- **成分表** = `build.json`：写明原料的 sha256、用什么火候（nvcc 参数）。
- **菜单** = `cake_jit.py` 的路由表：什么规格的订单上哪块蛋糕。

论文也说下游用户“不依赖 Cake”。追踪 issue 的原话是 “we will bake it for you”：你可以点单，但拿不到烤箱。

Cake IR：把硬件决策写成程序

本章的语言设计来自【论文】。仓库里没有 IR 源码，所以我用真实生成代码做对照，让你能从产物反推 IR 概念。

设计思路：描述 schedule，不是描述算法

【论文】 Cake IR 记录的是“机器该怎么被驱动”：哪些 warp 担任什么角色，哪些 buffer 分几级暂存，哪个 barrier 管哪次交接，哪种指令形式吃哪个操作数。分工的原则是：**schedule 说“要发生什么”，lowering 推导“怎么做”**。barrier 地址、phase 位、TMEM 偏移、描述符编码、warp 编号，都由声明算出来，agent 不用手写。

论文 Figure 3 的片段，简化如下：

```
@cake.schedule()
def fmha_fwd(lm, Q: LM.tma3d, O: LM.tma2d, seqlen_q: LM.i32):
    pool = lm.smem(98304)                              # 声明资源，不写裸地址
    smem_q = pool.view(offset=0, shape=(128, 128), dtype=lm.bf16, stage=3)
    tmem_acc = lm.tmem(cols=0, width=128, shape=(128, 128), dtype=lm.f32)

    load, mma = lm.role(warps=[0]), lm.role(warps=[1])  # 声明角色
    pipe = lm.pipeline(stages=3)
    q_full = lm.barrier(count=3, prod=[load], cons=[mma], init_count=1, pipeline=pipe)

    with load:
        for stage in lm.range(0, 3):
            smem_q.tma_load(Q, coords=(0, 0, stage), stage=stage, barrier=q_full)
    with mma:
        for stage in lm.range(0, 3):
            lm.wait(q_full, stage=stage)
            lm.fence_proxy()
            lm.mma(tmem_acc, smem_q[stage], smem_q[stage], init=(stage == 0))
```

简化自 CAKE 论文 Figure 3。

这段程序有四个性质撑起了后面的分析：

性质	含义
类型检查的词汇表	计算、内存移动、同步、warp 控制都是固定词汇，不能嵌 C 或 PTX
声明式资源	内存区、同步状态、流水线只声明一次，IR 知道每个 buffer 的形状、类型、寿命
显式角色	warp 组有名字，跨角色交接都看得见
自动推导元数据	声明的机械性后果由 lowering 算出

一个反直觉的决定：不要 layout

【论文】 近年的编译器都把 layout 做成代数对象：CuTe 的 layout 代数、Triton 的 F2 线性 layout、Axe 的命名轴。CAKE 走相反的路：**layout 不是 IR 的一等公民**。agent 直接写下具体承诺（SMEM 视图偏移、操作数字节偏移、TMEM 列范围、swizzle 标记、TMA 描述符坐标），由编译器判断这些承诺沿数据流是否一致、是否满足指令约束。

取舍很清楚：放弃表达力上的优雅，换来**编辑界面具体、错误容易定位**。这是为 agent 而不是为人设计的。

语言	硬件控制	layout 负担	对 agent 的问题
Triton、cuTile 等 tile DSL	被隐藏	低	写不出专家级 schedule
CuTe DSL	完全暴露	高，要掌握 layout 代数	易错，错误难定位
裸 CUDA + PTX	完全暴露	手算	结构不可分析
Cake IR	显式	写具体值，编译器验证	词汇表要持续扩充

先认识这些词

读生成代码需要这几个硬件术语：

术语	含义
CTA / cluster	一个 thread block。cluster 是可以共享内存、协同工作的一组 CTA
warp	32 个线程。warp 分工就是让不同 warp 干不同的活：搬数据、发 MMA、写结果
SMEM	CTA 内的共享内存，也是数据进入张量核之前的暂存区
TMEM	Blackwell 新增的张量内存，存放累加结果，按列分配
TMA	张量内存加速器，由一个线程发起的异步批量搬运
mbarrier	SMEM 里的异步 barrier，用 phase 位区分轮次
MMA	张量核的矩阵乘指令。Blackwell 上是 `tcgen05.mma`
PDL	可编程依赖 launch：后一个 kernel 不必等前一个完全结束就能排上

从生成代码反推 IR

下面用一个真实程序对照：NVFP4 SVDQuant GEMM 的 `05bda08 这个 kernel`。**codegen 把 IR 里的名字原样留在了注释和变量名里**，所以它是观察 IR 的好窗口。

IR 概念（论文）	生成代码里的样子	位置
`lm.role(...)`	`// ---- Role: mma ----` 和 `if (warp == 0)`	L438
`lm.pipeline` + `lm.barrier`	`// --- pipeline 'tma_pipe' ---`，接一串 `mbarrier_init`	L373
`lm.smem` 视图	`smem_a = smem_raw + 1024`，偏移已算好	L338
`lm.tmem`	`// TMEM alloc (128 columns, 80 used)`，接 `tcgen05.alloc`	L418
`tma_load(..., barrier=)`	`mbarrier_arrive_expect_tx` + `cp.async.bulk.tensor.3d···`	L543
`lm.wait` + `fence_proxy`	`mbarrier_wait_token` + `tcgen05.fence::after_thread_sync`	L451
`lm.mma`	`tcgen05_mma_mxf4nvf4_bs(tmem_accum, ···)`	L470
phase 位（自动推导）	`if (mma_stage == 15) { mma_stage = 0; _phase_tma_full ^= 1; }`	L478

这个 kernel 有五个角色：`mma`、`scheduler`（这里空闲）、`load`、`epilogue_load`、`epilogue`。它用 `__cluster_dims__(1,2,1)` 声明两个 CTA 组成一个 cluster（`L307`），数据可以用 multicast 同时写进两个 CTA。

一次 barrier 握手

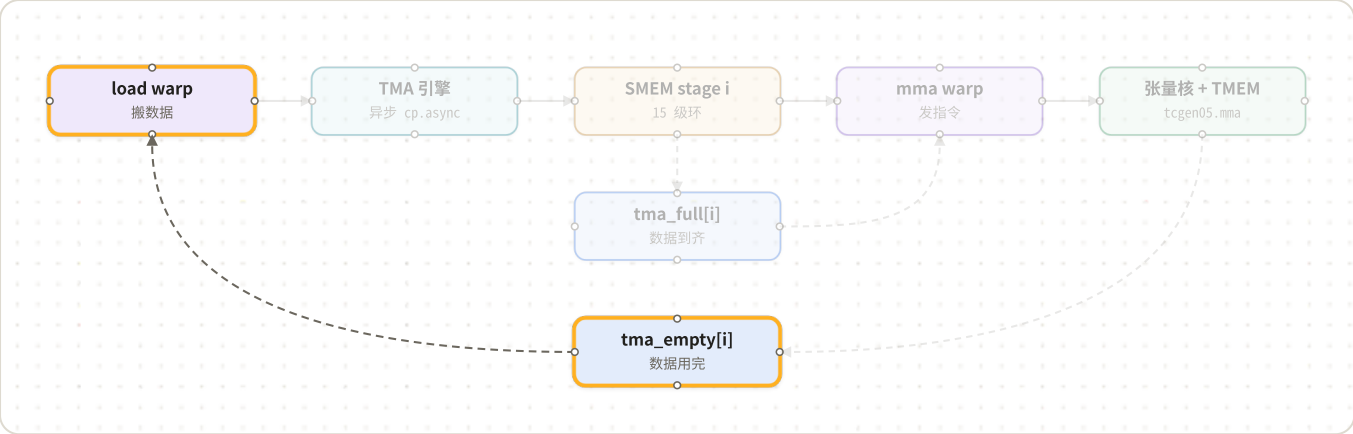
这是 warp 分工 kernel 的核心模式，也是手写时最容易出错的地方。这个 kernel 的 `tma_pipe` 有 15 个 stage，每个 stage 一对 barrier。

类比 · 餐厅的传菜口

- **传菜口的 15 个窗位** = SMEM 里的 15 个 pipeline stage。
- **厨师** = load 角色，用 TMA 异步把数据从全局内存搬进 SMEM。
- **服务员** = mma 角色，取走数据交给张量核。
- **窗位上的“有菜”灯** = `tma_full` barrier。
- **窗位上的“空了”灯** = `tma_empty` barrier。
- **灯的颜色每一圈翻转一次** = phase 位，所以第二圈不会把第一圈的旧信号当成新的。

双方都不用互相等待：厨师可以连做几道，服务员也可以连端几道，只要窗位够用。这就是异步流水线的收益。

▶ tma_pipe 的一个 stage 1 / 7



1. 等窗位空出来

load 等 `tma_empty[i]`。它的初始 phase 取 1（`L526`），所以第一圈不用真的等。

2. 声明要到多少字节

选出一个线程执行 `mbarrier_arrive_expect_tx(tma_full[i], 10240)`，告诉 barrier：这一轮有 10240 字节在路上（`L543`）。

3. 发起异步搬运

A、B 和两组 scale factor 都用 `cp.async.bulk.tensor` 发出。A 用 multicast 写进 cluster 里的两个 CTA（`L547`）。load 不等它完成，直接进入下一个 stage。

4. 数据到齐

TMA 完成时按字节数自动 arrive。mma 用 `mbarrier_wait_token` 等 `tma_full[i]`，它之前已经 `try_wait` 过一次，这里复用那个结果（`L451`）。

5. 发 MMA

先 `tcgen05.fence::after_thread_sync`，把 scale factor 拷进 TMEM，再由 elect 出的单个线程发 `tcgen05.mma`（`L470`）。

6. 归还窗位

`elect_commit_cg1_multicast(tma_empty[i], 3)` 在张量核完成后通知 cluster 里的两个 CTA：这个 stage 空了（`L476`）。

7. 绕回

`mma_stage` 到 15 归零，`_phase_tma_full` 翻转（`L478`）。这些机械细节就是论文说的“由 lowering 推导，agent 不写”的部分。

设计原则和架构覆盖

【论文】 IR 的演化受八条原则约束：

编号	原则	含义
P1	Ergonomic	编辑体验接近 NumPy/PyTorch
P2	Performance-transparent	影响性能的硬件决策可见
P3	Canonical	一种操作只有一种规范写法
P4	Statically type-checked	类型规则约束 lowering
P5	Analysis-friendly	暴露静态分析需要的信息
P6	Test-gated	IR 改动要过 kernel 矩阵测试
P7	Analysis-consistent	改数据模型必须同步改分析
P8	Hardware-grounded	每个操作都写明硬件行为

论文的架构覆盖表从 A100 的 `sm_80` 一直列到 DGX Spark 的 `sm_121a`，中间有 `sm_89`、`sm_90a`、`sm_100a`、`sm_103a`、`sm_120a`。编译器要求**精确匹配目标**，不做向下替换。**【仓库】** 产物比论文走得更远，已经出现 `sm_107a`（Rubin）和 `sm_110a`，见 `TARGET_FLAGS`。

IR 自己是“长出来”的：论文说它没有预先设计的词汇表，而是从一批生产 kernel 开始，由 agent 反复提炼模式、修改 IR、移植验证（论文 2.1 节和附录 A）。

Harness 与进化循环

本章除特别标注外都来自【论文】。 论文明确表示只描述验证器的契约和覆盖范围，不公开内部表示和判定过程。

分析套件

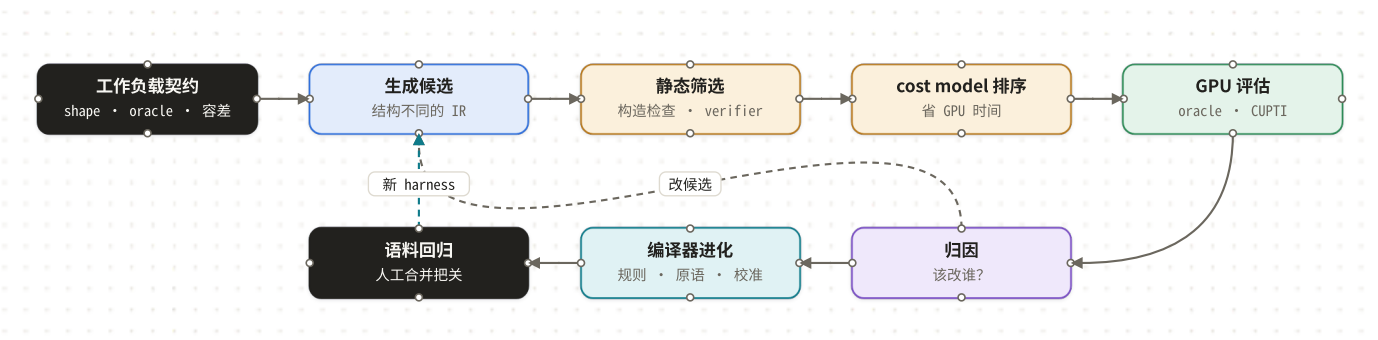
harness 就是 Cake IR 周围面向 agent 的环境。对外接口是一份契约：

类别	处置	作用
程序安全	编译前拦截	同步、顺序、内存使用风险
硬件一致性	编译前拦截	资源、指令、架构约束
数据一致性	编译前拦截	数据流和生产者/消费者表示是否兼容
调度语义	编译前拦截	schedule 的结构不变量
数值验证	执行时拦截	和外部权威参考对比输出
性能分析	报告	估算开销，归类瓶颈
优化建议	提示	不阻塞编译

分析是有边界的：它不证明全局 GPU 正确性，可能有误报和漏报；缺信息时显式报告，不当作通过。**GPU 上的实测才是最终依据。**

两层循环

内层进化 kernel，外层进化编译器



一次运行分四步：生成结构不同的候选，用便宜分析过滤，在 oracle 上评估幸存者，再把证据路由给候选、verifier、cost model 或 IR 词汇表。外层把“不透明的运行时崩溃”变成 verifier 规则，把“反复出现的非法 lowering”变成静态检查，把“系统性误判”变成校准目标。

论文有一个克制的设定：agent 和底层模型**固定不变**（实验都用 GPT-5.6-sol，xhigh），变的只有环境。这样提升才能归因到环境，而不是模型。**【仓库】** 追踪 issue 说 harness 配合 claude/codex agent 使用，所以实际生产用的 agent 不止一种。

实验结果

实验	结果	说明
Flash-KMeans 干净起步（B200，各 3 次，8000 万 token）	Cake IR 1.144×，裸 CUDA/PTX 0.928×	相对调优过的 FlashML 基线，取中位数
同上，达到 plateau	Cake IR 3/3，CUDA/PTX 0/3	活跃时长 1.89h 对 3.73h
KDA prefill	2.05× 几何平均（6 个 B200 shape）	对比官方 FlashKDA，通过 Kimi-K3 端到端 serving
KDA decode	1.14×	对比 upstream FlashInfer，30 个 shape
TinyGEMM	kernel 时间下降 18–23%	对比 TRT-LLM 派生实现，35 个 shape
已知 kernel 复现	11 项里 10 项达到或超过参考，最差 96.5%	Cake IR 行数都少于参考实现
KNN / KMeans 产品化	1.418×、2.116×、1.803×	共 434 个 shape，含 dispatcher 开销

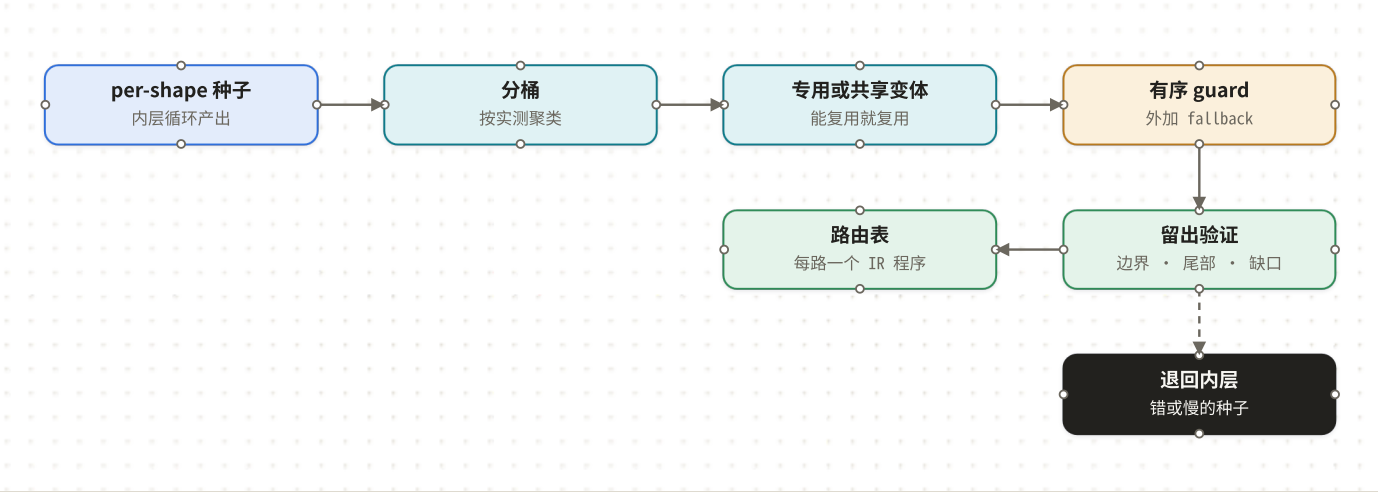
看这些数字时注意几点：干净起步实验样本只有每组 3 次；论文作者同时也是这些项目的作者；覆盖不均匀，大部分性能证据来自 B200，timing model 只校准了 B200 和 H100，其他架构会拒绝给出预测。

从一个 shape 到一个库

这一节很值得细看，因为它直接解释了仓库里的路由表为什么长成那样。

单个 shape 的优化可以很激进，但库要接受调用者传来的任何 shape。CAKE 把这当成**独立阶段**，目标函数、排序信号、失败模式都不同。

泛化阶段：把种子 kernel 变成 dispatcher



三条关键规则：评分包含 dispatcher 开销；有效 shape 域在调参前就声明好，dispatcher 的谓词只能划分它，不能新增方便的评估行；能复用一个物理 schedule 就复用，换 schedule 必须有实测收益。对应到 FlashInfer 就是下一章的 `ROUTES` 和 `PROGRAM_WINDOWS`。

论文 Figure 9 给了规模感：KNN build 的 112 个 shape 落到 8 个外层家族、80 条最终路由；Flash-KMeans 的 124 个 shape 落到 12 条路由。

产物怎么进仓库

难点

生成的代码量巨大，而且会频繁重新导出。FlashInfer 要解决四件事：能审计（知道每个文件从哪来）、能校验（防止被改坏）、不污染 lint、能让下游脱离 FlashInfer 使用。

五种交付形态

CAKE 进入仓库的方式演变过几轮，这几种现在**同时存在**。看到一个 CAKE kernel，先判断它属于哪种：

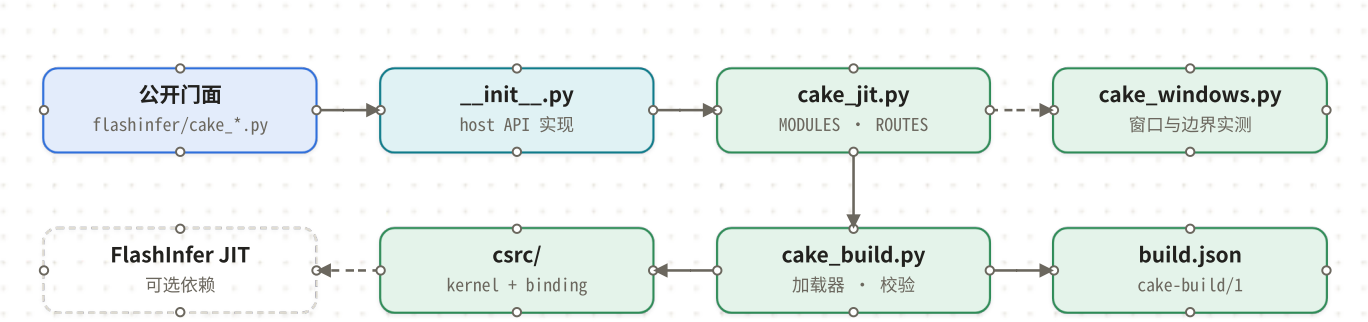
形态	特征	例子	能手改吗
A. 冻结源码	手写 loader，源文件头写着来源 commit	`softmax`，`loader`	已经改过，头部有记录
B. 生成的 in-tree loader	`flashinfer/jit/cake_<op>.py` 里一组 `CakeProgram` 常量	`CakeProgram`	否，exporter 字节级写入
C. 自包含包	一个目录：`build.json`、`cake_build.py`、`cake_jit.py`、`csrc/`	`rmsnorm`	否
D. 手工折叠	一份 manifest + 共享 kernel 体 + host 模板，靠 SASS 比对守住	`GDN`	是，但要验证 SASS 等价
E. 产品注册表	带能力矩阵和兜底路线的注册表	`cake_fmha`，`registry`	否

形态 D 值得单独说一下。GDN 的 README 第一段写明：kernel 由 Cake 生成，但现在的目录结构**由人维护**，靠逐个变体的 SASS 等价检查对照原始生成物（`README`）。这是目前仓库里“手工接管生成代码”最规范的先例。它甚至记录了一个细节：同样的宏值，写在文件内和通过 `-D` 传入，nvcc 13.4 在 sm_103a 上排出的指令序列不同（`README`）。

形态 C 的包结构

这是目前的主流形态，也是 #6553 的结果。提出这个 issue 的下游用 Bazel 预编译 CAKE 包，不依赖 FlashInfer 本身。在此之前，他们要维护约 1,040 行补丁和一个解析六种注册表格式的脚本。

一个 cake_*_backend 包的内部结构



后端的 init 文件 只依赖自身、`torch` 和 `tvm_ffi`。核心门面 只是几个 `@flashinfer_api` 的一行转发（注释解释了原因：公开 API 工具要读这里的签名）。

各文件的分工：

文件	内容	关键点
`build.json`	每个库的源文件、sha256、nvcc 参数、`min_cuda`、`closure_sha256`	**唯一**记录编译内容的地方
`cake_build.py`	优先加载 `aot/<name>.so`，否则走 JIT	所有包字节相同；预编译路径不 import flashinfer
`cake_jit.py`	`MODULES`、`ROUTES`、grid 规则、workspace 布局	只放 host 计划，不放编译信息
`cake_windows.py`	`PROGRAM_WINDOWS`、`EDGES`、文档片段	由注册表渲染，把实测结果也存下来
`csrc/*_kernel.cu`	设备代码，**单翻译单元**	在 rmsnorm 包里不 include `flashinfer/` 头文件
`csrc/*_binding.cu`	TVM-FFI launcher：校验、launch	只导出 `run` 和 `build_cuda_version`

内容寻址

文件名里的 20 位十六进制是内容摘要，这是整个方案的基础：

- `_verified_sources` 在编译前逐个校验源文件 sha256，不一致直接报错。
- `jit_spec` 把 `closure_sha256` 的前 20 位放进 spec 名字，闭包一变就不会命中旧缓存。
- 形态 B 的 `content_digest` 同样把源码字节和编译参数一起算进模块名。

所以手改一个生成文件的直接后果是：**下次加载就失败**。这是有意的 fail-closed（`docstring`）。

数值语义是契约的一部分

FlashInfer 的 JIT 默认开 `-use_fast_math`，CAKE 包的加载器显式关掉：`use_fast_math=False`。rmsnorm 包还逐库记录了 `--prec-div=true --prec-sqrt=true --ftz=false`。形态 B 还加 `--fmad=false`，文档的说法是“reproduces the numerics the sources were validated with”（`cake_program.py`）。

原因很直接：验证是针对特定浮点语义做的，编译参数变了，验证结果就不算数。接手 CAKE 代码时**不要统一编译参数**。

复用与 deslop

每个 kernel 一个翻译单元，意味着 helper 会被复制成千上万次。生成器后来把 helper 拆成内容寻址的头文件，放在 `cake_device_helpers/`（82 个），文件名是 `cake_<name>_<sha256前16位>.cuh`。

但代码膨胀还是严重。“deslop”是项目的术语，指一轮轮清理生成代码。#6635 记录的 `cake_mxfp4_situ_moe` 单个家族曾有 600 个文件、54 万行，其中 16 个 stage 根本不可达。常见手法：

手法	说明
删不可达路由	从注册表和 manifest 里移除
折叠孪生 kernel	如 `_nopdl` 版本：同一份源码，用宏区分，靠 sidecar 钉住被替换文件的摘要（`fold 测试`）
共享 binding	只差标识符的绑定合并成模板
提取 helper	进 `cake_device_helpers/`

> **注意 · 容易误解的地方：deslop 也会被冲掉**
>
> #7253 记录了一次真实回退：两次重新导出把 binding 文件从 64 个变回 102 个，因为新版生成物加了一个按 T 的检查，让之前共享的 binding 不再相同。只要编译器不变，**在仓库里做的清理就可能被下一次导出覆盖**。这也是为什么折叠后的文件头写着 “The Cake exporter rewrites this file; change its producer instead”（`fold 测试`）。

仓库如何放行生成代码

机制	做法	位置
pre-commit 例外	生成目录排除在格式化和 lint 之外	`exclude 规则`
目录级 `.clang-format`	让 clang-format 不动生成文件	`示例`
打包声明	每个包在 pyproject 登记 `csrc/**` 和 `build.json`；`aot/` 不随包发布	`pyproject`
清单检查	schema、摘要、引用闭合、加载器一致	`check_cake_build_manifests.py`

运行时：路由、窗口与调度规划

难点

kernel 是静态生成的，但调用时的 token 数是动态的，而且最优 schedule 随 token 数变化。host 代码要在**每次调用**的很短时间内选对程序、算对 grid、分好 workspace，还要兼容 CUDA Graph。

路由模型

以 `rmsnorm 的 cake_jit.py` 为例，它的开头注释把模型讲得很清楚：**一个 route 是一次公开调用可能 launch 的模块记录的有序列表**。一条模块记录是“一个 kernel + 一种 launch 策略”：

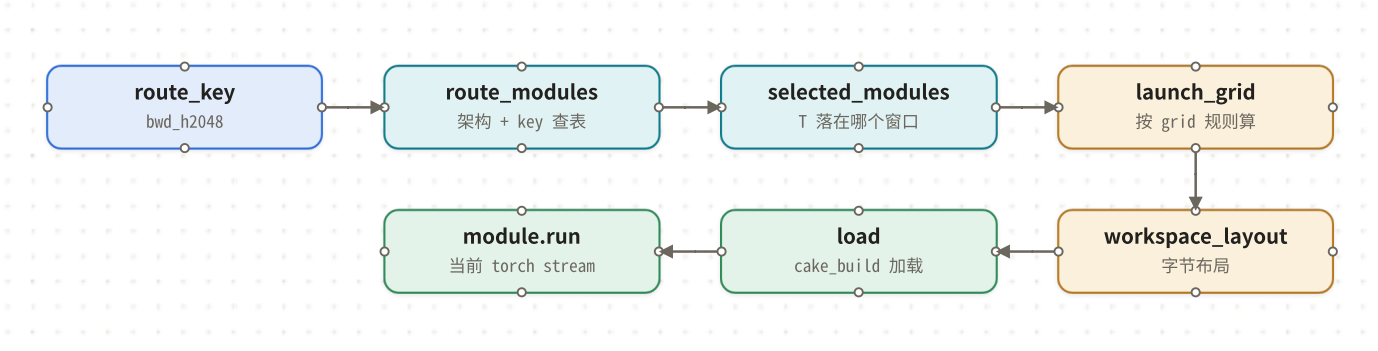
字段	含义
`kernel` / `library`	哪份源码、哪个 `.so` 词干
`arches`	交付给哪些架构
`role`	在路由里的角色：`fwd_small`、`bwd_main`、`bwd_reduce` …
`select_rule`	负责哪段 token 数：`rows_between(above, at_most)`
`grid_rule`	grid 怎么算：`rows_per_cta`、`fixed`、`claimed_chunks`
`launch`	`use_pdl`、`cooperative`
`workspace_rule`	需要多少 workspace，怎么布局

同一个 kernel 可以有多条记录，比如 `..._band` 这种后缀。它们指向同一个 `library`，所以只编译一次。这个包有 565 条模块记录，每个架构 32 条路由。

> **类比 · 变速箱的挡位**
>
> - **路由** = 一辆车的变速箱，按 `(hidden, fwd/bwd, residual, 权重类型)` 区分车型。
> - **窗口** = 挡位：`above < T ⩽ at_most`。
> - **程序** = 每个挡位的齿轮，低速扭矩大（小 T 用低延迟 kernel），高速省油（大 T 用吞吐 kernel）。
> - **换挡点** = 实测出来的 `EDGES`，还记录换挡时输出是否逐位相同。
>
> `program_windows` 会检查窗口是否恰好铺满所有正整数，不允许空挡或重叠。

选择路径是三个纯函数：

从调用参数到 launch



对应 `route_key`、`selected_modules`、`launch_grid`、`workspace_layout`、`run_module`。token 数是运行时标量，**换 token 数不触发重新编译**。

三种 grid 规则

规则	公式	用途
`rows_per_cta`	`ceil(rows / rows_per_cta)`，再按 `grid_limit` 或 `grid_per_sm × SM 数` 截断	前向，每 CTA 处理固定行数
`fixed`	固定 CTA 数	dw 的 reduce kernel，每列块一个 CTA
`claimed_chunks`	`min(slots, n_chunks + col_tasks)`，`slots = ctas_per_sm × SM 数`	反向主 kernel，持久 CTA 从计数器抢 chunk

注意 `sm_count` 是运行时从 `torch.cuda.get_device_properties` 读的（`sm_count`），所以同样的 kernel 在不同 SM 数的设备上，分块结果会不同。这也是“确定性”承诺要限定在相同 SM 数之内的原因。

host 和 kernel 还要计算同一张分块表：`chunk_rounds` 在 Python 里算一遍，kernel 开头用整数运算再算一遍（`kernel 入口附近`）。源码注释特意提醒：改这里的常量，就是改 kernel 文本（`TAIL_DIV 注释`）。

运行期 cost model

【仓库】 编译器的 cost model 不进仓库，但它的一部分结果以**校准常数**的形式进了 host 代码。选程序时有些决策用这些常数实时计算：

位置	决策	注释里的线索
`cake_ssd_combined.py`	Mamba SSD 选 exact scan 还是 chunk-parallel	从 seed 模块逐字复制，要一起刷新
`cake_fmha.py`	FMHA context 的 split 规划	微秒单位，按目标架构
`cake_kda_tf32_runtime.py`	KDA BF16 的 affine split	可用 `FLASHINFER_KDA_AFFINE_POLICY=legacy` 做 A/B

这些常数的注释常带着内部工单号和测量批次（`CAKE-956 round 5`）。如果你想在自己的硬件上调整，这里是**少数不需要编译器也能动的旋钮**。但要先确认对应的 kernel 和窗口没有被 exporter 覆盖。

产品化的入口：cake_fmha

`cake_fmha.py` 有约 8800 行，是路由思路最完整的例子。`文档` 给了几个关键事实：

- 它实现的是现有 TRT-LLM FMHA 的同一份契约，和 `trtllm_batch_decode_with_kv_cache` 同参同返回。
- 不加 `backend="cake"` 就不会走它。
- 注册表里有 1,906 个优化单元。
- 选择器未命中、workspace 不够或加载失败时，**fail closed 到** `compat_v1`，这个兼容组件覆盖完整定义域。

最后一点是库化的关键：路由表不管多大都不可能穷尽，所以必须有一条保底路线。

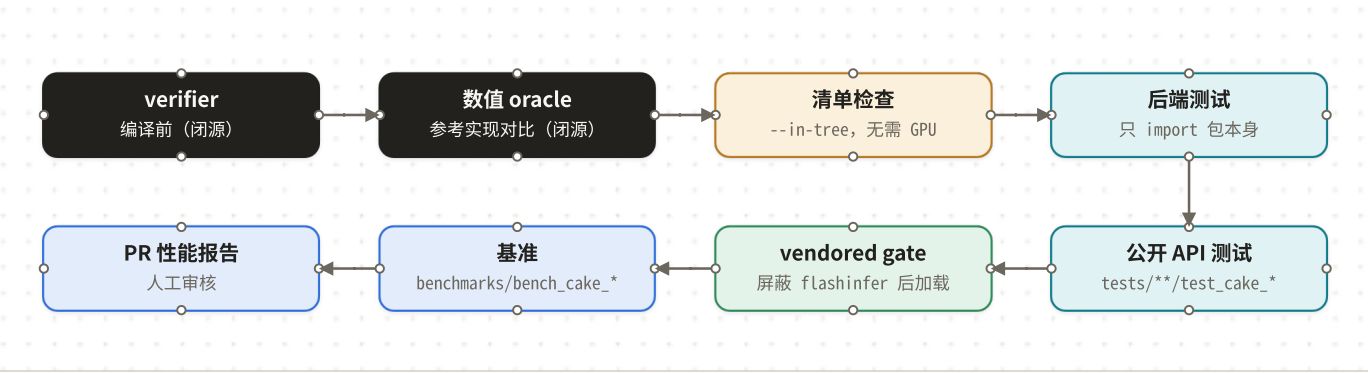
> **注意 · 容易误解的地方：SM 分区**
>
> 有些 `cake_fmha` 的小 M decode 规划要读 stream 所在的 SM 分区（`_smallm_launch_sm_count`）。`CLAUDE.md` 专门写了 `FLASHINFER_CAKE_FMHA_SM_PARTITIONS` 这个开关。用 green context 划分 SM 时，host 规划和实际 SM 数不一致会影响性能，这类问题 kernel 层看不出来。

验证与可信度

分层验证

生成代码没人逐行读，可信度只能来自验证。这是整个 CAKE 模式能成立的前提。

一段生成代码要过的关卡



上排前两关在编译器一侧，从第三关起都在仓库里。

各关卡的细节：

关卡	检查什么	位置
清单检查	schema、sha256、引用闭合、`cake_build.py` 字节一致、tvm-ffi 版本	`check_manifest`、`check_loader_identity`
vendored gate	拷贝包、只用清单编译、**用 meta-path finder 屏蔽 flashinfer** 后加载并跑测试	文档，`rmsnorm 测试`
包名隔离	后端测试通过一张表取包，方便下游指向自己的副本	`cake_package.py`
基准	62 个 CAKE 专用脚本，其中有的把“上一版 Cake”当基线	示例

确定性是一等概念

训练场景下 CAKE 把确定性做成了显式的产品维度，这是它和普通 kernel 库不同的地方：

模式	反向的 `dw` 怎么算	保证
`det`	main kernel 写每 chunk 的 FP32 partial 行，reduce kernel 按 chunk 顺序折叠	相同输入、硬件、计划下逐位可复现
`nondet`	持久 CTA 把 FP64 和按完成顺序累加，最后一个 CTA 落回 FP32	`dw` 不保证逐位可复现，`dx` 可以

`det` 的含义有明确边界：**不**承诺跨 shape、跨设备、跨并行划分结果相同（命名文档）。在部分 `nondet` 窗口里，TwoSum 补偿让误差不随 T 增长（`rmsnorm` 模块文档）。

和 PyTorch 的交互也有规定：`torch.use_deterministic_algorithms(True)` 下调用 `deterministic=False` 会报错；开 `warn_only=True` 时才回落（同上）。这部分正在改名，新规则是模式写进名字、不再带 `deterministic=` 参数，迁移逐族进行（命名规则）。所以现在同时能看到两种风格。

已知限制

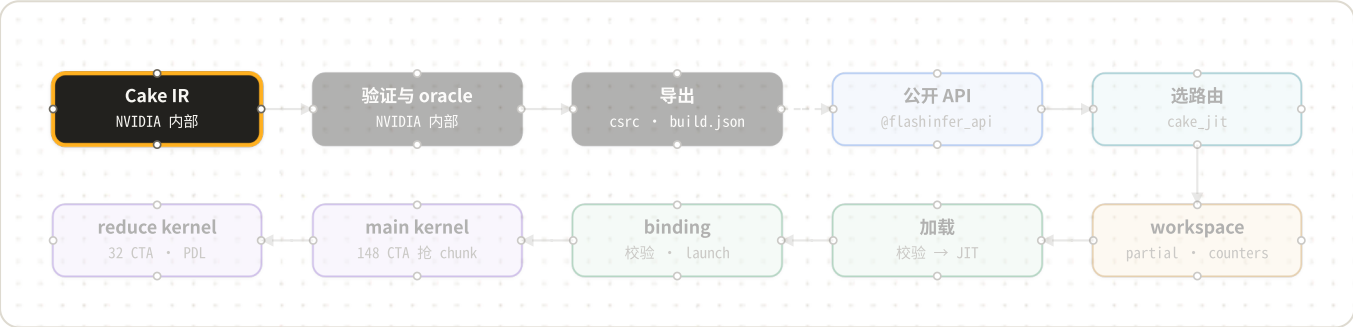
限制	来源
大部分性能证据来自 B200；timing model 只校准 B200 和 H100	【论文】第 8 节
静态分析不完整，有误报和漏报，不证明全局正确性	【论文】附录 C
编译器进化仍由人把关合并	【论文】第 8 节
非 NVIDIA 目标的迁移成本未测量，backend lowering 要重建	【论文】第 8 节
`check_cake_build_manifests.py` 的 `--built-wheel` 和 JIT 参数检查还没实现	【仓库】`TODO`
部分 CAKE 包通过实验轨道交付，没有兼容性保证	【仓库】实验声明
仓库里无法重新生成任何 kernel，也无法复现论文实验	【仓库】编译器不在其中

端到端生命周期

把前面的内容串起来：一个 RMSNorm 反向 kernel 从 IR 到 GPU。例子是 `cake_rmsnorm_train_backward(..., deterministic=False)`，B200（`sm_100a`，假设 148 个 SM），hidden 2048，T = 4096。

> **数字怎么来的**
>
> 路由和窗口是用 Python `ast` 从 `cake_jit.py` 静态解析字面量得到的。分块结果由我按 `chunk_rounds` 的逻辑重写计算，**没有运行仓库代码，也没有在 GPU 上跑过**。结果满足完整性校验：148×16 + 148×8 + 136×4 = 4096。

▶ 一次反向传播：前三步在编译器一侧，后面都在本仓库 1 / 10



1. Agent 写 schedule【不公开】

某个 agent 写出 Cake IR 程序，声明 warp 角色、SMEM、barrier。这份源码不在仓库里。

2. 验证【不公开】

verifier 拦截，cost model 排序，然后在 B200 上和参考实现比对。一批 shape 上的种子再泛化成路由（上一章的分桶流程）。

3. 导出

exporter 把程序写成 `_kernel.cu` 和 `_binding.cu`，并记录在 `build.json`。这个包的 `cake_revision` 是 `b10d84a`，2026-11-03 用 CUDA 13.4.92 渲染。

4. 调用公开 API

`cake_rmsnorm_train_backward` 转发到后端包。`_backward` 把输入整理成对齐的视图，检查路由是否适用。

5. 选路由

key 是 `bwd_h2048`。T = 4096 落在窗口 `3072 < T ⩽ 4096`，选中两条记录：主 kernel `9ac8d21e46a658e7913f`（角色 `bwd_mid`）和 reduce `880768a0a264e04dbfb5`。这个窗口虽然是 `deterministic=False`，走的却是确定性的 pair 形式：窗口决定形式，这正是注册表注释说的 “per token-count window”。

6. 算分块和 workspace

`claimed_chunks` 规则，`slots = 1 × 148`，`rows_per_issue = 4`。分块结果：148 个 16 行、148 个 8 行、136 个 4 行，共 432 个 chunk。主 kernel 的 grid 是 `min(148, 432) = 148`。workspace 由调用者持有，kernel 负责把计数器还原，所以 CUDA Graph 重放不需要 host 写入（`workspace_layout`）。

7. 加载

`load` 拼出库名 `..._sm_100a`。`load_library` 先找 `aot/`；没有就走 JIT：校验 sha256，用 `closure_sha256` 命名 spec，交给 FlashInfer 的 ninja 流程。这个库要求 CUDA 12.8 以上。

8. binding 校验并 launch

`binding` 校验 dtype、设备、连续性、i32 范围、grid 上限，然后 `cudaLaunchKernelExC`：`blockDim = 512`，动态 SMEM 132608 字节（`L136`）。

9. 主 kernel 执行

148 个 CTA 从 `counters` 用 `atomicAdd` 抢 chunk，大 chunk 先发，小 chunk 收尾，这是经典的负载均衡。每个 chunk 把一行 FP32 partial 写进 workspace，同时写 `dx`。

10. reduce 收尾

reduce kernel 用 32 个 CTA 按 chunk 顺序折叠 `dw`。它以 PDL 方式 launch：binding 设置 `cudaLaunchAttributeProgrammaticStreamSerialization`（`L77`），kernel 里用 `griddepcontrol.wait` 等前一个 grid（`L84`），减少两次 launch 之间的间隙。

这个例子能说明几件事。host 的 Python 只做规划，不碰数据；kernel 选择完全由数据表驱动；同一个 API 在 T 变化时可能换 kernel、换算法形式，甚至换精度路径，而调用者看不到。

开源许可证

各部分的许可证

部分	许可证	证据
FlashInfer 主体	Apache-2.0	`LICENSE`、`pyproject.toml`
CAKE 生成代码	文件头是 Apache-2.0	`kernel 文件头`
Hopper attention 部分文件	BSD 3-Clause（FlashAttention 派生）	`LICENSE 第三方段`
3rdparty/cutlass	BSD 3-Clause	`LICENSE.cutlass`
3rdparty/spdlog、fmt	MIT	`LICENSE.spdlog`
KDA 部分 kernel	MIT	`NOTICE`、`LICENSE.kda-for-kda`
vendor 的 cuDNN Frontend	MIT 和 Apache-2.0 混合，逐文件标注	`NOTICE`
CAKE 编译器	未公开，无从谈许可证	见下

规则和场景

Apache-2.0 是宽松许可证：可以商用、修改、闭源分发，要保留版权和 NOTICE，修改过的文件要标明，并包含专利授权和专利报复条款。

场景	允许吗	要做什么
公司内部用	允许	无
给上游提 PR	允许	我没有找到 CLA 或 DCO 要求；照 PR 模板写
公开 fork	允许	保留 LICENSE、NOTICE，标明改动
vendor CAKE 包进自己的产品	允许	带上 LICENSE 和 NOTICE；这也是 #6553 支持的用法
闭源 SaaS	允许	Apache-2.0 没有网络条款
发行二进制 wheel	允许	带上第三方许可证文本，见 `licenses/`
使用 CAKE 编译器	目前做不到	没有公开发布，不要当作可依赖的工具链

历史

用 GitHub API 和首个 commit 的文件核实：`LICENSE` 在 2023-07-22 的首次提交里就是 Apache-2.0，现在 GitHub 识别的也是 Apache-2.0。此后有 5 次提交改过这个文件，最近一次在 2026-08-31。从 commit 标题看都和新增第三方组件有关，我没有逐条 diff。

> **不是法律意见**
>
> 这里是工程师读代码时的理解。`NOTICE` 里列的第三方来源较多，真要分发请逐条核对 `NOTICE` 和对应目录的 `VENDOR.md`，并咨询律师。

贡献与定制

开发环境

```
git clone https://github.com/flashinfer-ai/flashinfer.git --recursive
cd flashinfer
pip install --no-build-isolation -e . -v
pre-commit install
pytest tests/path/test_file.py::test_function
```

`--recursive` 是必须的，子模块见 `.gitmodules`。`--no-build-isolation` 防止 pip 拉进不兼容的 PyTorch。`CLAUDE.md` 有 1200 多行，是事实上的开发手册，AGENTS.md 只是转到这里（`AGENTS.md`）。加 kernel 的步骤见 `add-cuda-kernel`。

项目的 review 规则

- 用默认 PR 模板，不要改写成自定义格式（`模板`）。标题和描述会变成 squash 后的 commit message。
- 性能 PR 必须给**绝对数值**，并写明 GPU 和 shape（`PR 规则`）。
- 欢迎 AI 辅助，但作者要能讲清设计，否则 PR 可能被拒。
- `include/` 不准出现 Torch 头文件。
- 新的公开 API 要配 `TraceTemplate`（`清单`）。
- 触发 CI 默认用定向测试：`@flashinfer-bot run tests/<路径>`，`run all` 很贵（`说明`）。
- 实验性功能要有 owner、tracking issue、毕业计划，默认四周内毕业。
- review 的重点在 `code_review_guidance.md`：越界、溢出、workspace 大小、同步。

碰 CAKE 代码时

想做的事	可以吗	做法
改 `*_kernel.cu`、`cake_build.py`、`cake_windows.py`	**不要**	sha 校验会失败，下次导出也会覆盖
改公开门面、文档、docstring	可以	门面是手写的
加测试、基准	可以	后端测试走 `cake_package.py`
修 host 逻辑的 bug	谨慎	`cake_jit.py` 的数据由 exporter 填写。动手前先看这个文件最近是不是被导出 PR 重写过，并在 PR 里写明
请求新 kernel	可以	在 #4254 留言或开 sub-issue；训练类看 #4642
清理重复代码	谨慎	学 deslop issue 的写法，先统计影响，并确认能经受重新导出
手工接管一个家族	可以但重	学 GDN：manifest + SASS 等价检查

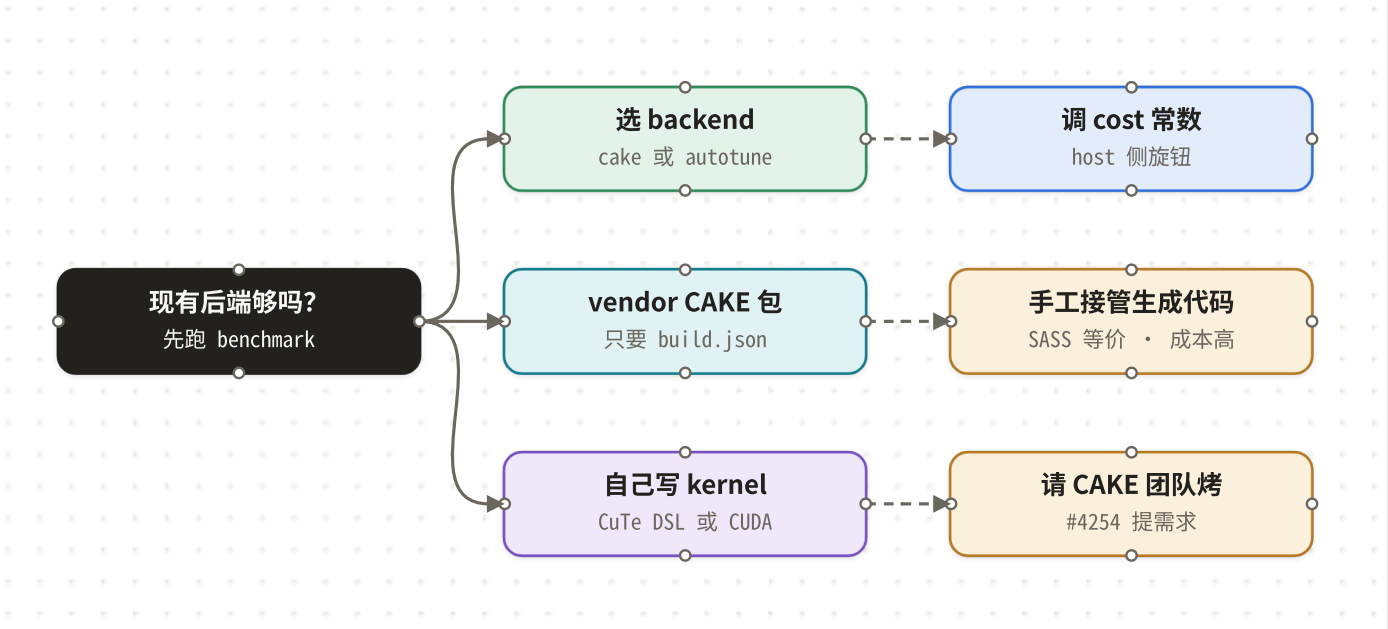
适合入手的方向

- **测试和边界条件**：路由窗口边缘、workspace 大小、CUDA Graph 重放。这类工作不依赖编译器，价值高。
- **手写 kernel 的小修复**：`include/` 下的 kernel 和 CUTLASS 路径。
- **文档**：很多 CAKE README 写得很细，但没有总览，这份导读能看出缺口。
- **fi_trace**：`@flashinfer_api` 自动提供 `.fi_trace()`，产出的 JSON 是 flashinfer-bench 的输入（`fi_trace 文档`）。

需要避开的方向：大范围改动生成代码、统一 CAKE 的编译参数、给 `backend="auto"` 增加实验后端的路径。

定制路线

我想要一个更快的 kernel，该走哪条路?



从上到下成本递增。vendor 流程的权威说明在 检查脚本文档 和 #6553。

路线	代价	依赖	适合
选 backend / autotune	低	无	先做这一步
调 host 校准常数	低	自己的基准	SM 数或负载和官方不同
vendor CAKE 包	中	`build.json`、nvcc、tvm-ffi	需要 hermetic 构建、镜像内预编译
写 CuTe DSL / CUDA	高	对硬件的深入理解	官方没有覆盖的算子
请求 CAKE kernel	不确定	上游排期	公开模型的热点算子
自建类似 Cake 的环境	很高	论文只给设计，不给实现	研究团队

走最后一条路之前，可以先看两个第三方复现：cake-ir（面向 CUDA）和 cake-ascend（面向华为 Ascend）。它们都是根据论文独立实现的，和官方 CAKE 没有关系，我只看了 README，没有评估过质量。

附录：源码索引

FlashInfer 主干

主题	链接
开发手册	`CLAUDE.md`、`CONTRIBUTING.md`
公开导出	`flashinfer/__init__.py`
CLI	`__main__.py`
JIT 核心	`core.py`、`cpp_ext.py`、`env.py`
构建后端	`build_backend.py`
AOT	`aot.py`、`artifacts.py`
backend 机制	`backend_requirement`、`experimental_backend`
实验策略	`README`、`CLAUDE.md`
Trace	`fi_trace.py`、`trace/`
CI	`pr-test.yml`、`Jenkinsfile`

CAKE 产物

主题	链接
形态 A 样本	`softmax 冻结源码`
形态 B 运行时	`cake_program.py`
形态 C 样本	`rmsnorm 后端包`
形态 D 样本	`GDN README`、`cake_gdn.py`
形态 E 样本	`cake_fmha.py`、`registry.json`
共享 helper	`cake_device_helpers/`
检查脚本	`check_cake_build_manifests.py`
观察 IR 的 kernel	`svdquant 05bda08`
命名规则	`training_backend_names.md`
家族 README	`DSv4`、`Kimi-K3 MLA`
后端测试入口	`cake_package.py`
教程	`Kimi-K3 SiTU`

仓库外的资料

- CAKE 论文（arXiv 2608.12629）
- CAKE kernel 进度追踪 #4254
- CAKE 包独立构建 #6553
- 首个 CAKE PR #4262
- FlashInfer 文档
- 第三方复现，非官方，未评估：cake-ir、cake-ascend